DATA DOWNLOAD & MANAGEMENT TOOLS GUIDE

AI-Based Monitoring Solution for Oil & Gas Industry

Document Version: 1.0 **Date**: December 2025 **Audience**: Developers, Data Engineers, ML Engineers

---

TABLE OF CONTENTS

1. Data Sources Overview
2. InfluxDB Data Operations
3. PostgreSQL Data Operations
4. MinIO/S3 Object Storage
5. Kafka Data Streaming
6. Data Export Tools
7. Data Import Tools
8. Data Transformation
9. Backup & Restore
10. Complete Workflows

---

1. DATA SOURCES OVERVIEW

1.1 Data Architecture

```
DATA SOURCES & STORAGE
======================

EDGE LAYER                    STREAMING                    STORAGE
==========                    =========                    =======

Sensors -----> Edge Gateway -----> Kafka -----> InfluxDB (Time-Series)
                    |                  |              |
                    |                  +--------> PostgreSQL (Metadata)
                    |                  |              |
Cameras -----> Video Analytics       +--------> MinIO (Raw Files)
                    |                                 |
                    +-------------------------------> Milvus (Vectors)
```

1.2 Connection Credentials

```bash
# Create credentials file
cat > ~/.oilgas-credentials << 'EOF'
# InfluxDB
export INFLUXDB_URL="http://influxdb.oilgas.internal:8086"
export INFLUXDB_ORG="oilgas"
export INFLUXDB_TOKEN="your-influxdb-token"
export INFLUXDB_BUCKET="sensor_data"

# PostgreSQL
export POSTGRES_HOST="postgres.oilgas.internal"
export POSTGRES_PORT="5432"
```

```
export POSTGRES_DB="oilgas_ai"
export POSTGRES_USER="developer"
export POSTGRES_PASSWORD="your-postgres-password"
export DATABASE_URL="postgresql://${POSTGRES_USER}:${POSTGRES_PASSWORD}@${POSTGRES_HOST}:${POSTGRES_POR

# MinIO/S3
export MINIO_ENDPOINT="minio.oilgas.internal:9000"
export MINIO_ACCESS_KEY="your-access-key"
export MINIO_SECRET_KEY="your-secret-key"
export AWS_ACCESS_KEY_ID="${MINIO_ACCESS_KEY}"
export AWS_SECRET_ACCESS_KEY="${MINIO_SECRET_KEY}"

# Kafka
export KAFKA_BOOTSTRAP_SERVERS="kafka.oilgas.internal:9092"

# Milvus
export MILVUS_HOST="milvus.oilgas.internal"
export MILVUS_PORT="19530"
EOF

# Load credentials
source ~/.oilgas-credentials
```

---

2. INFLUXDB DATA OPERATIONS

2.1 Installation & Setup

```
# Install InfluxDB CLI
wget https://dl.influxdata.com/influxdb/releases/influxdb2-client-2.7.3-linux-amd64.tar.gz
tar xvzf influxdb2-client-2.7.3-linux-amd64.tar.gz
sudo mv influx /usr/local/bin/

# Configure CLI
influx config create \
  --config-name oilgas \
  --host-url $INFLUXDB_URL \
  --org $INFLUXDB_ORG \
  --token $INFLUXDB_TOKEN \
  --active

# Verify connection
influx ping
influx bucket list
```

2.2 Query Sensor Data

```
# List available measurements
influx query 'import "influxdata/influxdb/schema"
schema.measurements(bucket: "sensor_data")'

# Query latest sensor readings
influx query '
```

```
from(bucket: "sensor_data")
  |> range(start: -1h)
  |> filter(fn: (r) => r._measurement == "sensor_readings")
  |> filter(fn: (r) => r.asset_id == "PUMP-001")
  |> last()
'

# Query time range with aggregation
influx query '
from(bucket: "sensor_data")
  |> range(start: -24h)
  |> filter(fn: (r) => r._measurement == "sensor_readings")
  |> filter(fn: (r) => r.sensor_type == "vibration")
  |> aggregateWindow(every: 1h, fn: mean)
  |> yield(name: "hourly_average")
'

# Export to CSV
influx query '
from(bucket: "sensor_data")
  |> range(start: -7d)
  |> filter(fn: (r) => r._measurement == "vibration_features")
  |> filter(fn: (r) => r.asset_id == "PUMP-001")
' --raw > vibration_data.csv
```

2.3 Python InfluxDB Client

```
#!/usr/bin/env python3
# scripts/influxdb_download.py
"""
InfluxDB Data Download Script

Usage:
    python influxdb_download.py --asset PUMP-001 --start 2025-01-01 --end 2025-01-31 --output data/
"""

import os
import argparse
from datetime import datetime
from pathlib import Path
import pandas as pd
from influxdb_client import InfluxDBClient
from influxdb_client.client.write_api import SYNCHRONOUS
from loguru import logger

class InfluxDBDownloader:
    """Download data from InfluxDB"""

    def __init__(self):
        self.url = os.environ.get('INFLUXDB_URL', 'http://localhost:8086')
        self.token = os.environ.get('INFLUXDB_TOKEN')
        self.org = os.environ.get('INFLUXDB_ORG', 'oilgas')

        self.client = InfluxDBClient(
```

```
            url=self.url,
            token=self.token,
            org=self.org
        )
        self.query_api = self.client.query_api()

    def download_sensor_readings(
        self,
        asset_id: str,
        start_time: str,
        end_time: str,
        bucket: str = "sensor_data"
    ) -> pd.DataFrame:
        """
        Download sensor readings for an asset.

        Args:
            asset_id: Asset identifier (e.g., "PUMP-001")
            start_time: Start time (ISO format or relative like "-7d")
            end_time: End time (ISO format or "now()")
            bucket: InfluxDB bucket name

        Returns:
            DataFrame with sensor readings
        """
        query = f'''
        from(bucket: "{bucket}")
          |> range(start: {start_time}, stop: {end_time})
          |> filter(fn: (r) => r._measurement == "sensor_readings")
          |> filter(fn: (r) => r.asset_id == "{asset_id}")
          |> pivot(rowKey:["_time"], columnKey: ["_field"], valueColumn: "_value")
        '''

        logger.info(f"Downloading sensor data for {asset_id}")
        result = self.query_api.query_data_frame(query)

        if isinstance(result, list):
            result = pd.concat(result, ignore_index=True)

        logger.info(f"Downloaded {len(result)} records")
        return result

    def download_vibration_features(
        self,
        asset_id: str,
        start_time: str,
        end_time: str
    ) -> pd.DataFrame:
        """Download pre-computed vibration features"""
        query = f'''
        from(bucket: "features")
          |> range(start: {start_time}, stop: {end_time})
          |> filter(fn: (r) => r._measurement == "vibration_features")
          |> filter(fn: (r) => r.asset_id == "{asset_id}")
```

```
            |> pivot(rowKey:["_time"], columnKey: ["_field"], valueColumn: "_value")
        '''

        logger.info(f"Downloading vibration features for {asset_id}")
        result = self.query_api.query_data_frame(query)

        if isinstance(result, list):
            result = pd.concat(result, ignore_index=True)

        return result

    def download_predictions(
        self,
        asset_id: str,
        start_time: str,
        end_time: str
    ) -> pd.DataFrame:
        """Download ML predictions"""
        query = f'''
        from(bucket: "predictions")
          |> range(start: {start_time}, stop: {end_time})
          |> filter(fn: (r) => r.asset_id == "{asset_id}")
          |> pivot(rowKey:["_time"], columnKey: ["_field"], valueColumn: "_value")
        '''

        return self.query_api.query_data_frame(query)

    def download_raw_waveforms(
        self,
        asset_id: str,
        start_time: str,
        end_time: str,
        limit: int = 100
    ) -> pd.DataFrame:
        """Download raw vibration waveforms"""
        query = f'''
        from(bucket: "raw_data")
          |> range(start: {start_time}, stop: {end_time})
          |> filter(fn: (r) => r._measurement == "vibration_waveform")
          |> filter(fn: (r) => r.asset_id == "{asset_id}")
          |> limit(n: {limit})
        '''

        return self.query_api.query_data_frame(query)

    def list_assets(self) -> list:
        """List all available assets"""
        query = '''
        import "influxdata/influxdb/schema"
        schema.tagValues(bucket: "sensor_data", tag: "asset_id")
        '''
        result = self.query_api.query(query)
        return [record.get_value() for table in result for record in table.records]
```

```
    def get_data_summary(self, asset_id: str) -> dict:
        """Get data availability summary for an asset"""
        query = f'''
        from(bucket: "sensor_data")
          |> range(start: -365d)
          |> filter(fn: (r) => r.asset_id == "{asset_id}")
          |> group()
          |> count()
        '''
        result = self.query_api.query(query)
        count = result[0].records[0].get_value() if result else 0

        # Get time range
        first_query = f'''
        from(bucket: "sensor_data")
          |> range(start: -365d)
          |> filter(fn: (r) => r.asset_id == "{asset_id}")
          |> first()
        '''
        last_query = f'''
        from(bucket: "sensor_data")
          |> range(start: -365d)
          |> filter(fn: (r) => r.asset_id == "{asset_id}")
          |> last()
        '''

        return {
            'asset_id': asset_id,
            'record_count': count,
            'buckets': ['sensor_data', 'features', 'predictions']
        }


def main():
    parser = argparse.ArgumentParser(description='Download data from InfluxDB')
    parser.add_argument('--asset', required=True, help='Asset ID')
    parser.add_argument('--start', required=True, help='Start time (ISO or relative)')
    parser.add_argument('--end', default='now()', help='End time')
    parser.add_argument('--output', default='./data', help='Output directory')
    parser.add_argument('--format', choices=['csv', 'parquet'], default='parquet')
    parser.add_argument('--type', choices=['readings', 'features', 'predictions', 'all'],
                        default='all', help='Data type to download')

    args = parser.parse_args()

    # Create output directory
    output_dir = Path(args.output)
    output_dir.mkdir(parents=True, exist_ok=True)

    # Initialize downloader
    downloader = InfluxDBDownloader()

    # Download data
    if args.type in ['readings', 'all']:
```

```
            df = downloader.download_sensor_readings(args.asset, args.start, args.end)
            if not df.empty:
                output_file = output_dir / f"{args.asset}_readings.{args.format}"
                if args.format == 'csv':
                    df.to_csv(output_file, index=False)
                else:
                    df.to_parquet(output_file, index=False)
                logger.info(f"Saved to {output_file}")

        if args.type in ['features', 'all']:
            df = downloader.download_vibration_features(args.asset, args.start, args.end)
            if not df.empty:
                output_file = output_dir / f"{args.asset}_features.{args.format}"
                if args.format == 'csv':
                    df.to_csv(output_file, index=False)
                else:
                    df.to_parquet(output_file, index=False)
                logger.info(f"Saved to {output_file}")

        if args.type in ['predictions', 'all']:
            df = downloader.download_predictions(args.asset, args.start, args.end)
            if not df.empty:
                output_file = output_dir / f"{args.asset}_predictions.{args.format}"
                if args.format == 'csv':
                    df.to_csv(output_file, index=False)
                else:
                    df.to_parquet(output_file, index=False)
                logger.info(f"Saved to {output_file}")


if __name__ == '__main__':
    main()
```

2.4 Bulk Data Export

```
#!/bin/bash
# scripts/bulk_export_influxdb.sh
# Bulk export data from InfluxDB for all assets

set -e

OUTPUT_DIR="${1:-./exported_data}"
START_DATE="${2:--30d}"
END_DATE="${3:-now()}"

mkdir -p "$OUTPUT_DIR"

echo "Exporting InfluxDB data to $OUTPUT_DIR"
echo "Time range: $START_DATE to $END_DATE"

# Get list of assets
ASSETS=$(influx query 'import "influxdata/influxdb/schema"
schema.tagValues(bucket: "sensor_data", tag: "asset_id")' --raw | tail -n +2)
```

```
for asset in $ASSETS; do
    echo "Exporting data for asset: $asset"

    # Export sensor readings
    influx query "
    from(bucket: \"sensor_data\")
      |> range(start: $START_DATE, stop: $END_DATE)
      |> filter(fn: (r) => r.asset_id == \"$asset\")
      |> pivot(rowKey:[\"_time\"], columnKey: [\"_field\"], valueColumn: \"_value\")
    " --raw > "$OUTPUT_DIR/${asset}_sensor_data.csv"

    # Export features
    influx query "
    from(bucket: \"features\")
      |> range(start: $START_DATE, stop: $END_DATE)
      |> filter(fn: (r) => r.asset_id == \"$asset\")
      |> pivot(rowKey:[\"_time\"], columnKey: [\"_field\"], valueColumn: \"_value\")
    " --raw > "$OUTPUT_DIR/${asset}_features.csv"

    echo "  - Exported to ${asset}_sensor_data.csv and ${asset}_features.csv"
done

# Create manifest
echo "Creating manifest..."
cat > "$OUTPUT_DIR/manifest.json" << EOF
{
  "export_date": "$(date -Iseconds)",
  "time_range": {
    "start": "$START_DATE",
    "end": "$END_DATE"
  },
  "assets": [$(echo $ASSETS | sed 's/ /", "/g' | sed 's/^/"/' | sed 's/$/"/')],
  "files": $(ls -1 "$OUTPUT_DIR"/*.csv | jq -R -s -c 'split("\n") | map(select(. != ""))')
}
EOF

echo "Export complete. Total files: $(ls -1 "$OUTPUT_DIR"/*.csv | wc -l)"
```

---

3. POSTGRESQL DATA OPERATIONS

3.1 Installation & Setup

```
# Install PostgreSQL client
sudo apt-get update
sudo apt-get install -y postgresql-client-16

# Install pgcli (enhanced CLI)
pip install pgcli

# Test connection
psql "postgresql://$POSTGRES_USER:$POSTGRES_PASSWORD@$POSTGRES_HOST:$POSTGRES_PORT/$POSTGRES_DB" -c "SE
```

```
# Or use pgcli
pgcli "postgresql://$POSTGRES_USER:$POSTGRES_PASSWORD@$POSTGRES_HOST:$POSTGRES_PORT/$POSTGRES_DB"
```

3.2 Query Operations

```
-- Connect to database
-- psql -h $POSTGRES_HOST -U $POSTGRES_USER -d $POSTGRES_DB

-- List all schemas
\dn

-- List tables in schema
\dt asset_mgmt.*
\dt alerts.*
\dt ml_models.*
\dt knowledge.*

-- Describe table structure
\d+ asset_mgmt.assets

-- Query assets with sensors
SELECT
    a.asset_id,
    a.asset_tag,
    a.name,
    a.asset_type,
    a.status,
    COUNT(s.sensor_id) as sensor_count
FROM asset_mgmt.assets a
LEFT JOIN asset_mgmt.sensors s ON a.asset_id = s.asset_id
GROUP BY a.asset_id
ORDER BY a.asset_tag;

-- Query active alerts
SELECT
    al.alert_id,
    a.asset_tag,
    ad.alert_name,
    ad.severity,
    al.triggered_at,
    al.status
FROM alerts.alert_instances al
JOIN alerts.alert_definitions ad ON al.definition_id = ad.definition_id
JOIN asset_mgmt.assets a ON al.asset_id = a.asset_id
WHERE al.status = 'active'
ORDER BY al.triggered_at DESC
LIMIT 50;

-- Query ML model deployments
SELECT
    m.model_name,
    m.model_version,
    d.deployment_target,
    d.deployed_at,
```

```
    d.status,
    m.metrics
FROM ml_models.model_registry m
JOIN ml_models.model_deployments d ON m.model_id = d.model_id
WHERE d.status = 'active'
ORDER BY d.deployed_at DESC;

-- Query prediction history
SELECT
    p.prediction_id,
    a.asset_tag,
    p.prediction_type,
    p.predicted_value,
    p.confidence,
    p.prediction_time
FROM ml_models.prediction_log p
JOIN asset_mgmt.assets a ON p.asset_id = a.asset_id
WHERE p.prediction_time > NOW() - INTERVAL '24 hours'
ORDER BY p.prediction_time DESC
LIMIT 100;

-- Export query results to CSV
\copy (SELECT * FROM asset_mgmt.assets) TO '/tmp/assets.csv' WITH CSV HEADER;
```

3.3 Python PostgreSQL Client

```
#!/usr/bin/env python3
# scripts/postgres_download.py
"""
PostgreSQL Data Download Script

Usage:
    python postgres_download.py --table assets --output data/assets.csv
    python postgres_download.py --query "SELECT * FROM asset_mgmt.assets WHERE status='operational'" --
"""

import os
import argparse
from pathlib import Path
import pandas as pd
from sqlalchemy import create_engine, text
from loguru import logger

class PostgresDownloader:
    """Download data from PostgreSQL"""

    def __init__(self):
        self.database_url = os.environ.get('DATABASE_URL')
        if not self.database_url:
            host = os.environ.get('POSTGRES_HOST', 'localhost')
            port = os.environ.get('POSTGRES_PORT', '5432')
            db = os.environ.get('POSTGRES_DB', 'oilgas_ai')
            user = os.environ.get('POSTGRES_USER', 'developer')
            password = os.environ.get('POSTGRES_PASSWORD', '')
```

```
            self.database_url = f"postgresql://{user}:{password}@{host}:{port}/{db}"

        self.engine = create_engine(self.database_url)

    def list_tables(self, schema: str = None) -> list:
        """List all tables in database or schema"""
        if schema:
            query = f"""
            SELECT table_name
            FROM information_schema.tables
            WHERE table_schema = '{schema}'
            ORDER BY table_name
            """
        else:
            query = """
            SELECT table_schema, table_name
            FROM information_schema.tables
            WHERE table_schema NOT IN ('pg_catalog', 'information_schema')
            ORDER BY table_schema, table_name
            """
        return pd.read_sql(query, self.engine)

    def download_table(
        self,
        table_name: str,
        schema: str = None,
        limit: int = None,
        where: str = None
    ) -> pd.DataFrame:
        """
        Download entire table or filtered subset.

        Args:
            table_name: Table name
            schema: Schema name (optional)
            limit: Maximum rows to download
            where: WHERE clause condition
        """
        full_table = f"{schema}.{table_name}" if schema else table_name

        query = f"SELECT * FROM {full_table}"
        if where:
            query += f" WHERE {where}"
        if limit:
            query += f" LIMIT {limit}"

        logger.info(f"Downloading from {full_table}")
        df = pd.read_sql(query, self.engine)
        logger.info(f"Downloaded {len(df)} rows")
        return df

    def execute_query(self, query: str) -> pd.DataFrame:
        """Execute custom SQL query"""
        logger.info(f"Executing query: {query[:100]}...")
```

```
    return pd.read_sql(query, self.engine)

def download_assets_with_sensors(self) -> pd.DataFrame:
    """Download assets with their sensors"""
    query = """
    SELECT
        a.asset_id,
        a.asset_tag,
        a.name as asset_name,
        a.asset_type,
        a.status,
        a.criticality,
        l.name as location_name,
        s.sensor_id,
        s.sensor_tag,
        s.sensor_type,
        s.unit,
        s.min_value,
        s.max_value
    FROM asset_mgmt.assets a
    LEFT JOIN asset_mgmt.locations l ON a.location_id = l.location_id
    LEFT JOIN asset_mgmt.sensors s ON a.asset_id = s.asset_id
    ORDER BY a.asset_tag, s.sensor_tag
    """
    return pd.read_sql(query, self.engine)

def download_alerts_history(
    self,
    start_date: str = None,
    end_date: str = None,
    severity: str = None
) -> pd.DataFrame:
    """Download alert history"""
    query = """
    SELECT
        ai.alert_id,
        a.asset_tag,
        a.name as asset_name,
        ad.alert_name,
        ad.severity,
        ad.condition_type,
        ai.triggered_value,
        ai.triggered_at,
        ai.acknowledged_at,
        ai.resolved_at,
        ai.status,
        ai.notes
    FROM alerts.alert_instances ai
    JOIN alerts.alert_definitions ad ON ai.definition_id = ad.definition_id
    JOIN asset_mgmt.assets a ON ai.asset_id = a.asset_id
    WHERE 1=1
    """

    if start_date:
```

```
            query += f" AND ai.triggered_at >= '{start_date}'"
        if end_date:
            query += f" AND ai.triggered_at <= '{end_date}'"
        if severity:
            query += f" AND ad.severity = '{severity}'"

        query += " ORDER BY ai.triggered_at DESC"

        return pd.read_sql(query, self.engine)

    def download_ml_predictions(
        self,
        asset_id: str = None,
        prediction_type: str = None,
        days: int = 30
    ) -> pd.DataFrame:
        """Download ML prediction logs"""
        query = f"""
        SELECT
            p.prediction_id,
            a.asset_tag,
            m.model_name,
            m.model_version,
            p.prediction_type,
            p.input_features,
            p.predicted_value,
            p.confidence,
            p.prediction_time
        FROM ml_models.prediction_log p
        JOIN asset_mgmt.assets a ON p.asset_id = a.asset_id
        JOIN ml_models.model_registry m ON p.model_id = m.model_id
        WHERE p.prediction_time > NOW() - INTERVAL '{days} days'
        """

        if asset_id:
            query += f" AND p.asset_id = '{asset_id}'"
        if prediction_type:
            query += f" AND p.prediction_type = '{prediction_type}'"

        query += " ORDER BY p.prediction_time DESC"

        return pd.read_sql(query, self.engine)

    def download_knowledge_documents(self) -> pd.DataFrame:
        """Download document metadata from knowledge base"""
        query = """
        SELECT
            d.document_id,
            d.file_name,
            d.file_type,
            d.file_size_bytes,
            d.indexed_at,
            d.status,
            COUNT(c.chunk_id) as chunk_count
```

```
        FROM knowledge.documents d
        LEFT JOIN knowledge.chunks c ON d.document_id = c.document_id
        GROUP BY d.document_id
        ORDER BY d.indexed_at DESC
        """
        return pd.read_sql(query, self.engine)


def main():
    parser = argparse.ArgumentParser(description='Download data from PostgreSQL')
    parser.add_argument('--table', help='Table name to download')
    parser.add_argument('--schema', help='Schema name')
    parser.add_argument('--query', help='Custom SQL query')
    parser.add_argument('--output', required=True, help='Output file or directory')
    parser.add_argument('--format', choices=['csv', 'parquet', 'json'], default='csv')
    parser.add_argument('--limit', type=int, help='Limit rows')
    parser.add_argument('--where', help='WHERE clause')
    parser.add_argument('--preset', choices=['assets', 'alerts', 'predictions', 'documents'],
                        help='Use preset query')

    args = parser.parse_args()

    downloader = PostgresDownloader()

    # Execute based on arguments
    if args.preset:
        if args.preset == 'assets':
            df = downloader.download_assets_with_sensors()
        elif args.preset == 'alerts':
            df = downloader.download_alerts_history()
        elif args.preset == 'predictions':
            df = downloader.download_ml_predictions()
        elif args.preset == 'documents':
            df = downloader.download_knowledge_documents()
    elif args.query:
        df = downloader.execute_query(args.query)
    elif args.table:
        df = downloader.download_table(args.table, args.schema, args.limit, args.where)
    else:
        # List tables
        df = downloader.list_tables(args.schema)
        print(df.to_string())
        return

    # Save output
    output_path = Path(args.output)
    if output_path.is_dir():
        filename = args.preset or args.table or 'query_result'
        output_path = output_path / f"{filename}.{args.format}"

    if args.format == 'csv':
        df.to_csv(output_path, index=False)
    elif args.format == 'parquet':
        df.to_parquet(output_path, index=False)
```

```
    elif args.format == 'json':
        df.to_json(output_path, orient='records', indent=2)

    logger.info(f"Saved {len(df)} rows to {output_path}")


if __name__ == '__main__':
    main()
```

3.4 Database Dump & Restore

```
#!/bin/bash
# scripts/postgres_backup.sh
# PostgreSQL backup and restore operations

# Full database dump
pg_dump -h $POSTGRES_HOST -U $POSTGRES_USER -d $POSTGRES_DB \
    -Fc -f backup_$(date +%Y%m%d).dump

# Schema-only dump
pg_dump -h $POSTGRES_HOST -U $POSTGRES_USER -d $POSTGRES_DB \
    --schema-only -f schema_$(date +%Y%m%d).sql

# Data-only dump
pg_dump -h $POSTGRES_HOST -U $POSTGRES_USER -d $POSTGRES_DB \
    --data-only -f data_$(date +%Y%m%d).sql

# Dump specific schema
pg_dump -h $POSTGRES_HOST -U $POSTGRES_USER -d $POSTGRES_DB \
    -n asset_mgmt -Fc -f asset_mgmt_$(date +%Y%m%d).dump

# Dump specific table
pg_dump -h $POSTGRES_HOST -U $POSTGRES_USER -d $POSTGRES_DB \
    -t asset_mgmt.assets -Fc -f assets_$(date +%Y%m%d).dump

# Restore full database
pg_restore -h $POSTGRES_HOST -U $POSTGRES_USER -d $POSTGRES_DB \
    -c backup_20251210.dump

# Restore specific schema
pg_restore -h $POSTGRES_HOST -U $POSTGRES_USER -d $POSTGRES_DB \
    -n asset_mgmt asset_mgmt_20251210.dump
```

---

4. MINIO/S3 OBJECT STORAGE

4.1 Installation & Setup

```
# Install MinIO client
wget https://dl.min.io/client/mc/release/linux-amd64/mc
chmod +x mc
sudo mv mc /usr/local/bin/
```

```
# Configure MinIO client
mc alias set oilgas http://$MINIO_ENDPOINT $MINIO_ACCESS_KEY $MINIO_SECRET_KEY

# Verify connection
mc admin info oilgas

# List buckets
mc ls oilgas

# Alternative: Use AWS CLI
pip install awscli

aws configure set aws_access_key_id $MINIO_ACCESS_KEY
aws configure set aws_secret_access_key $MINIO_SECRET_KEY

# Use with endpoint
aws --endpoint-url http://$MINIO_ENDPOINT s3 ls
```

4.2 Bucket Operations

```
# Create buckets
mc mb oilgas/ml-data
mc mb oilgas/raw-sensor-data
mc mb oilgas/model-artifacts
mc mb oilgas/documents

# List bucket contents
mc ls oilgas/ml-data
mc ls oilgas/ml-data/training/ --recursive

# Get bucket info
mc stat oilgas/ml-data

# Set bucket policy (public read)
mc anonymous set download oilgas/documents

# Set bucket policy (private)
mc anonymous set none oilgas/ml-data
```

4.3 File Operations

```
# Upload file
mc cp local_file.csv oilgas/ml-data/datasets/

# Upload directory
mc cp --recursive ./training_data/ oilgas/ml-data/training/

# Download file
mc cp oilgas/ml-data/models/anomaly_detector.pt ./models/

# Download directory
mc cp --recursive oilgas/ml-data/training/ ./local_training/

# Mirror (sync) directories
```

```
mc mirror ./local_data/ oilgas/ml-data/
mc mirror oilgas/ml-data/ ./local_backup/

# Find files
mc find oilgas/ml-data --name "*.parquet"
mc find oilgas/ml-data --larger 100MB

# Remove files
mc rm oilgas/ml-data/old_file.csv
mc rm --recursive --force oilgas/ml-data/temp/
```

4.4 Python MinIO Client

```
#!/usr/bin/env python3
# scripts/minio_operations.py
"""
MinIO/S3 Data Operations

Usage:
    python minio_operations.py upload --bucket ml-data --file model.pt --path models/
    python minio_operations.py download --bucket ml-data --prefix training/ --output ./data/
    python minio_operations.py list --bucket ml-data
"""

import os
import argparse
from pathlib import Path
from minio import Minio
from minio.error import S3Error
from loguru import logger
import io

class MinIOClient:
    """MinIO/S3 operations client"""

    def __init__(self):
        self.endpoint = os.environ.get('MINIO_ENDPOINT', 'localhost:9000')
        self.access_key = os.environ.get('MINIO_ACCESS_KEY')
        self.secret_key = os.environ.get('MINIO_SECRET_KEY')

        self.client = Minio(
            self.endpoint,
            access_key=self.access_key,
            secret_key=self.secret_key,
            secure=False  # Set True for HTTPS
        )

    def list_buckets(self) -> list:
        """List all buckets"""
        buckets = self.client.list_buckets()
        return [b.name for b in buckets]

    def create_bucket(self, bucket_name: str):
        """Create a bucket if not exists"""
```

```
    if not self.client.bucket_exists(bucket_name):
        self.client.make_bucket(bucket_name)
        logger.info(f"Created bucket: {bucket_name}")
    else:
        logger.info(f"Bucket already exists: {bucket_name}")

def list_objects(
    self,
    bucket: str,
    prefix: str = "",
    recursive: bool = True
) -> list:
    """List objects in bucket"""
    objects = self.client.list_objects(
        bucket,
        prefix=prefix,
        recursive=recursive
    )
    return [
        {
            'name': obj.object_name,
            'size': obj.size,
            'modified': obj.last_modified
        }
        for obj in objects
    ]

def upload_file(
    self,
    bucket: str,
    file_path: str,
    object_name: str = None
):
    """Upload a file to bucket"""
    if object_name is None:
        object_name = Path(file_path).name

    self.client.fput_object(bucket, object_name, file_path)
    logger.info(f"Uploaded {file_path} to {bucket}/{object_name}")

def upload_directory(
    self,
    bucket: str,
    local_dir: str,
    prefix: str = ""
):
    """Upload entire directory"""
    local_path = Path(local_dir)

    for file_path in local_path.rglob('*'):
        if file_path.is_file():
            relative_path = file_path.relative_to(local_path)
            object_name = f"{prefix}/{relative_path}" if prefix else str(relative_path)
            self.upload_file(bucket, str(file_path), object_name)
```

```python
def download_file(
    self,
    bucket: str,
    object_name: str,
    file_path: str
):
    """Download a file from bucket"""
    Path(file_path).parent.mkdir(parents=True, exist_ok=True)
    self.client.fget_object(bucket, object_name, file_path)
    logger.info(f"Downloaded {bucket}/{object_name} to {file_path}")

def download_directory(
    self,
    bucket: str,
    prefix: str,
    local_dir: str
):
    """Download all objects with prefix"""
    objects = self.list_objects(bucket, prefix)

    for obj in objects:
        relative_path = obj['name']
        if prefix:
            relative_path = obj['name'][len(prefix):].lstrip('/')

        local_path = Path(local_dir) / relative_path
        self.download_file(bucket, obj['name'], str(local_path))

def upload_dataframe(
    self,
    bucket: str,
    df,
    object_name: str,
    format: str = 'parquet'
):
    """Upload pandas DataFrame directly"""
    buffer = io.BytesIO()

    if format == 'parquet':
        df.to_parquet(buffer, index=False)
    elif format == 'csv':
        df.to_csv(buffer, index=False)

    buffer.seek(0)
    size = buffer.getbuffer().nbytes

    self.client.put_object(
        bucket,
        object_name,
        buffer,
        length=size,
        content_type='application/octet-stream'
    )
```

```
        logger.info(f"Uploaded DataFrame to {bucket}/{object_name}")

    def download_dataframe(
        self,
        bucket: str,
        object_name: str,
        format: str = 'parquet'
    ):
        """Download object as pandas DataFrame"""
        import pandas as pd

        response = self.client.get_object(bucket, object_name)
        buffer = io.BytesIO(response.read())

        if format == 'parquet':
            return pd.read_parquet(buffer)
        elif format == 'csv':
            return pd.read_csv(buffer)

    def get_presigned_url(
        self,
        bucket: str,
        object_name: str,
        expiry_hours: int = 24
    ) -> str:
        """Generate presigned URL for temporary access"""
        from datetime import timedelta

        url = self.client.presigned_get_object(
            bucket,
            object_name,
            expires=timedelta(hours=expiry_hours)
        )
        return url


def main():
    parser = argparse.ArgumentParser(description='MinIO/S3 operations')
    subparsers = parser.add_subparsers(dest='command', required=True)

    # List command
    list_parser = subparsers.add_parser('list', help='List objects')
    list_parser.add_argument('--bucket', required=True)
    list_parser.add_argument('--prefix', default='')

    # Upload command
    upload_parser = subparsers.add_parser('upload', help='Upload files')
    upload_parser.add_argument('--bucket', required=True)
    upload_parser.add_argument('--file', help='File to upload')
    upload_parser.add_argument('--dir', help='Directory to upload')
    upload_parser.add_argument('--path', default='', help='Target path prefix')

    # Download command
    download_parser = subparsers.add_parser('download', help='Download files')
```

```
    download_parser.add_argument('--bucket', required=True)
    download_parser.add_argument('--object', help='Object name')
    download_parser.add_argument('--prefix', help='Prefix for bulk download')
    download_parser.add_argument('--output', required=True, help='Output path')

    args = parser.parse_args()

    client = MinIOClient()

    if args.command == 'list':
        objects = client.list_objects(args.bucket, args.prefix)
        for obj in objects:
            print(f"{obj['size']:>12} {obj['modified']} {obj['name']}")

    elif args.command == 'upload':
        if args.file:
            object_name = f"{args.path}/{Path(args.file).name}" if args.path else None
            client.upload_file(args.bucket, args.file, object_name)
        elif args.dir:
            client.upload_directory(args.bucket, args.dir, args.path)

    elif args.command == 'download':
        if args.object:
            client.download_file(args.bucket, args.object, args.output)
        elif args.prefix:
            client.download_directory(args.bucket, args.prefix, args.output)


if __name__ == '__main__':
    main()
```

---

5. KAFKA DATA STREAMING

5.1 Installation & Setup

```
# Install Kafka CLI tools
wget https://downloads.apache.org/kafka/3.6.1/kafka_2.13-3.6.1.tgz
tar xzf kafka_2.13-3.6.1.tgz
export PATH=$PATH:$(pwd)/kafka_2.13-3.6.1/bin

# Or use kcat (formerly kafkacat)
sudo apt-get install -y kafkacat

# Test connection
kcat -b $KAFKA_BOOTSTRAP_SERVERS -L
```

5.2 Topic Operations

```
# List topics
kafka-topics.sh --bootstrap-server $KAFKA_BOOTSTRAP_SERVERS --list

# Create topic
```

```
kafka-topics.sh --bootstrap-server $KAFKA_BOOTSTRAP_SERVERS \
    --create --topic sensor_readings \
    --partitions 10 \
    --replication-factor 3

# Describe topic
kafka-topics.sh --bootstrap-server $KAFKA_BOOTSTRAP_SERVERS \
    --describe --topic sensor_readings

# Delete topic
kafka-topics.sh --bootstrap-server $KAFKA_BOOTSTRAP_SERVERS \
    --delete --topic old_topic

# Alter topic partitions
kafka-topics.sh --bootstrap-server $KAFKA_BOOTSTRAP_SERVERS \
    --alter --topic sensor_readings \
    --partitions 20
```

5.3 Consume & Produce Messages

```
# Consume messages (from beginning)
kafka-console-consumer.sh --bootstrap-server $KAFKA_BOOTSTRAP_SERVERS \
    --topic sensor_readings \
    --from-beginning \
    --max-messages 100

# Consume with key
kafka-console-consumer.sh --bootstrap-server $KAFKA_BOOTSTRAP_SERVERS \
    --topic sensor_readings \
    --property print.key=true \
    --property key.separator=": "

# Consume from specific partition/offset
kafka-console-consumer.sh --bootstrap-server $KAFKA_BOOTSTRAP_SERVERS \
    --topic sensor_readings \
    --partition 0 \
    --offset 1000

# Produce messages
echo '{"sensor_id": "SENS-001", "value": 42.5}' | \
kafka-console-producer.sh --bootstrap-server $KAFKA_BOOTSTRAP_SERVERS \
    --topic sensor_readings

# Using kcat
kcat -b $KAFKA_BOOTSTRAP_SERVERS -t sensor_readings -C -c 100
kcat -b $KAFKA_BOOTSTRAP_SERVERS -t sensor_readings -P -l messages.txt
```

5.4 Python Kafka Client

```
#!/usr/bin/env python3
# scripts/kafka_operations.py
"""
Kafka Data Operations
```

```
Usage:
    python kafka_operations.py consume --topic sensor_readings --output data/messages.json --count 1000
    python kafka_operations.py produce --topic sensor_readings --file data/messages.json
"""

import os
import json
import argparse
from datetime import datetime
from pathlib import Path
from confluent_kafka import Consumer, Producer, KafkaError, KafkaException
from confluent_kafka.admin import AdminClient, NewTopic
from loguru import logger

class KafkaClient:
    """Kafka operations client"""

    def __init__(self):
        self.bootstrap_servers = os.environ.get(
            'KAFKA_BOOTSTRAP_SERVERS',
            'localhost:9092'
        )
        self.admin_config = {'bootstrap.servers': self.bootstrap_servers}

    def list_topics(self) -> list:
        """List all topics"""
        admin = AdminClient(self.admin_config)
        metadata = admin.list_topics(timeout=10)
        return list(metadata.topics.keys())

    def create_topic(
        self,
        topic: str,
        partitions: int = 10,
        replication_factor: int = 3
    ):
        """Create a new topic"""
        admin = AdminClient(self.admin_config)
        new_topic = NewTopic(
            topic,
            num_partitions=partitions,
            replication_factor=replication_factor
        )

        futures = admin.create_topics([new_topic])
        for topic, future in futures.items():
            try:
                future.result()
                logger.info(f"Created topic: {topic}")
            except Exception as e:
                logger.error(f"Failed to create topic {topic}: {e}")

    def consume_messages(
        self,
```

```
    topic: str,
    group_id: str = None,
    count: int = None,
    timeout: float = 10.0,
    from_beginning: bool = True
) -> list:
    """
    Consume messages from topic.

    Args:
        topic: Topic name
        group_id: Consumer group ID
        count: Number of messages to consume (None = unlimited)
        timeout: Timeout in seconds
        from_beginning: Start from beginning if True
    """
    config = {
        'bootstrap.servers': self.bootstrap_servers,
        'group.id': group_id or f'downloader-{datetime.now().timestamp()}',
        'auto.offset.reset': 'earliest' if from_beginning else 'latest',
        'enable.auto.commit': False
    }

    consumer = Consumer(config)
    consumer.subscribe([topic])

    messages = []
    try:
        while True:
            msg = consumer.poll(timeout)

            if msg is None:
                if count is None or len(messages) >= count:
                    break
                continue

            if msg.error():
                if msg.error().code() == KafkaError._PARTITION_EOF:
                    break
                raise KafkaException(msg.error())

            message_data = {
                'topic': msg.topic(),
                'partition': msg.partition(),
                'offset': msg.offset(),
                'timestamp': msg.timestamp()[1],
                'key': msg.key().decode('utf-8') if msg.key() else None,
                'value': json.loads(msg.value().decode('utf-8'))
            }
            messages.append(message_data)

            if count and len(messages) >= count:
                break
```

```
        finally:
            consumer.close()

        logger.info(f"Consumed {len(messages)} messages from {topic}")
        return messages

    def produce_messages(
        self,
        topic: str,
        messages: list,
        key_field: str = None
    ):
        """
        Produce messages to topic.

        Args:
            topic: Topic name
            messages: List of message dictionaries
            key_field: Field to use as message key
        """
        config = {
            'bootstrap.servers': self.bootstrap_servers,
            'client.id': 'python-producer'
        }

        producer = Producer(config)
        delivered = 0

        def delivery_callback(err, msg):
            nonlocal delivered
            if err:
                logger.error(f"Delivery failed: {err}")
            else:
                delivered += 1

        for msg in messages:
            key = None
            if key_field and key_field in msg:
                key = str(msg[key_field]).encode('utf-8')

            value = json.dumps(msg).encode('utf-8')
            producer.produce(topic, value=value, key=key, callback=delivery_callback)
            producer.poll(0)

        producer.flush()
        logger.info(f"Produced {delivered} messages to {topic}")

    def get_consumer_lag(self, group_id: str, topic: str) -> dict:
        """Get consumer group lag"""
        from confluent_kafka.admin import AdminClient

        admin = AdminClient(self.admin_config)

        # Get committed offsets
```

```
        consumer_config = {
            'bootstrap.servers': self.bootstrap_servers,
            'group.id': group_id
        }
        consumer = Consumer(consumer_config)

        # Get topic partitions
        metadata = admin.list_topics(topic)
        partitions = metadata.topics[topic].partitions

        lag_info = {}
        for partition_id in partitions:
            from confluent_kafka import TopicPartition
            tp = TopicPartition(topic, partition_id)

            # Get committed offset
            committed = consumer.committed([tp])[0]
            committed_offset = committed.offset if committed else 0

            # Get end offset (high watermark)
            low, high = consumer.get_watermark_offsets(tp)

            lag_info[partition_id] = {
                'committed': committed_offset,
                'end': high,
                'lag': high - committed_offset
            }

        consumer.close()
        return lag_info


def main():
    parser = argparse.ArgumentParser(description='Kafka operations')
    subparsers = parser.add_subparsers(dest='command', required=True)

    # List command
    list_parser = subparsers.add_parser('list', help='List topics')

    # Consume command
    consume_parser = subparsers.add_parser('consume', help='Consume messages')
    consume_parser.add_argument('--topic', required=True)
    consume_parser.add_argument('--count', type=int, help='Number of messages')
    consume_parser.add_argument('--output', required=True, help='Output file')
    consume_parser.add_argument('--group', help='Consumer group ID')

    # Produce command
    produce_parser = subparsers.add_parser('produce', help='Produce messages')
    produce_parser.add_argument('--topic', required=True)
    produce_parser.add_argument('--file', required=True, help='JSON file with messages')
    produce_parser.add_argument('--key-field', help='Field to use as key')

    args = parser.parse_args()
```

```
    client = KafkaClient()

    if args.command == 'list':
        topics = client.list_topics()
        for topic in sorted(topics):
            print(topic)

    elif args.command == 'consume':
        messages = client.consume_messages(
            args.topic,
            group_id=args.group,
            count=args.count
        )

        output_path = Path(args.output)
        output_path.parent.mkdir(parents=True, exist_ok=True)

        with open(output_path, 'w') as f:
            json.dump(messages, f, indent=2, default=str)

        logger.info(f"Saved {len(messages)} messages to {output_path}")

    elif args.command == 'produce':
        with open(args.file) as f:
            messages = json.load(f)

        client.produce_messages(args.topic, messages, args.key_field)


if __name__ == '__main__':
    main()
```

---

6. DATA EXPORT TOOLS

6.1 Unified Export Script

```
#!/usr/bin/env python3
# scripts/unified_export.py
"""
Unified Data Export Tool

Exports data from all sources to a unified format for ML training.

Usage:
    python unified_export.py --asset PUMP-001 --start 2025-01-01 --end 2025-01-31 --output ./training_d
    python unified_export.py --all-assets --days 30 --output ./bulk_export/
"""

import os
import argparse
from datetime import datetime, timedelta
from pathlib import Path
```

```python
import pandas as pd
from loguru import logger
import json

# Import our data clients
from influxdb_download import InfluxDBDownloader
from postgres_download import PostgresDownloader
from minio_operations import MinIOClient


class UnifiedExporter:
    """Export data from all sources"""

    def __init__(self, output_dir: str):
        self.output_dir = Path(output_dir)
        self.output_dir.mkdir(parents=True, exist_ok=True)

        self.influx = InfluxDBDownloader()
        self.postgres = PostgresDownloader()
        self.minio = MinIOClient()

        self.manifest = {
            'export_timestamp': datetime.utcnow().isoformat(),
            'assets': [],
            'files': []
        }

    def export_asset(
        self,
        asset_id: str,
        start_time: str,
        end_time: str
    ) -> dict:
        """Export all data for a single asset"""
        asset_dir = self.output_dir / asset_id
        asset_dir.mkdir(exist_ok=True)

        export_info = {
            'asset_id': asset_id,
            'time_range': {'start': start_time, 'end': end_time},
            'files': []
        }

        # 1. Export sensor readings from InfluxDB
        logger.info(f"Exporting sensor readings for {asset_id}")
        try:
            df_readings = self.influx.download_sensor_readings(
                asset_id, start_time, end_time
            )
            if not df_readings.empty:
                file_path = asset_dir / 'sensor_readings.parquet'
                df_readings.to_parquet(file_path, index=False)
                export_info['files'].append({
                    'type': 'sensor_readings',
```

```
                'path': str(file_path.relative_to(self.output_dir)),
                'rows': len(df_readings)
            })
    except Exception as e:
        logger.error(f"Failed to export sensor readings: {e}")

    # 2. Export vibration features from InfluxDB
    logger.info(f"Exporting vibration features for {asset_id}")
    try:
        df_features = self.influx.download_vibration_features(
            asset_id, start_time, end_time
        )
        if not df_features.empty:
            file_path = asset_dir / 'vibration_features.parquet'
            df_features.to_parquet(file_path, index=False)
            export_info['files'].append({
                'type': 'vibration_features',
                'path': str(file_path.relative_to(self.output_dir)),
                'rows': len(df_features)
            })
    except Exception as e:
        logger.error(f"Failed to export vibration features: {e}")

    # 3. Export predictions from InfluxDB
    logger.info(f"Exporting predictions for {asset_id}")
    try:
        df_predictions = self.influx.download_predictions(
            asset_id, start_time, end_time
        )
        if not df_predictions.empty:
            file_path = asset_dir / 'predictions.parquet'
            df_predictions.to_parquet(file_path, index=False)
            export_info['files'].append({
                'type': 'predictions',
                'path': str(file_path.relative_to(self.output_dir)),
                'rows': len(df_predictions)
            })
    except Exception as e:
        logger.error(f"Failed to export predictions: {e}")

    # 4. Export asset metadata from PostgreSQL
    logger.info(f"Exporting metadata for {asset_id}")
    try:
        query = f"""
        SELECT a.*, l.name as location_name
        FROM asset_mgmt.assets a
        LEFT JOIN asset_mgmt.locations l ON a.location_id = l.location_id
        WHERE a.asset_id = '{asset_id}' OR a.asset_tag = '{asset_id}'
        """
        df_metadata = self.postgres.execute_query(query)
        if not df_metadata.empty:
            file_path = asset_dir / 'metadata.json'
            df_metadata.to_json(file_path, orient='records', indent=2)
            export_info['files'].append({
```

```
                'type': 'metadata',
                'path': str(file_path.relative_to(self.output_dir)),
                'rows': len(df_metadata)
            })
    except Exception as e:
        logger.error(f"Failed to export metadata: {e}")

    # 5. Export alert history from PostgreSQL
    logger.info(f"Exporting alerts for {asset_id}")
    try:
        query = f"""
        SELECT ai.*, ad.alert_name, ad.severity
        FROM alerts.alert_instances ai
        JOIN alerts.alert_definitions ad ON ai.definition_id = ad.definition_id
        WHERE ai.asset_id = '{asset_id}'
        AND ai.triggered_at BETWEEN '{start_time}' AND '{end_time}'
        ORDER BY ai.triggered_at
        """
        df_alerts = self.postgres.execute_query(query)
        if not df_alerts.empty:
            file_path = asset_dir / 'alerts.parquet'
            df_alerts.to_parquet(file_path, index=False)
            export_info['files'].append({
                'type': 'alerts',
                'path': str(file_path.relative_to(self.output_dir)),
                'rows': len(df_alerts)
            })
    except Exception as e:
        logger.error(f"Failed to export alerts: {e}")

    return export_info

def export_all_assets(self, days: int = 30):
    """Export data for all assets"""
    end_time = 'now()'
    start_time = f'-{days}d'

    # Get asset list
    assets = self.influx.list_assets()
    logger.info(f"Found {len(assets)} assets to export")

    for asset_id in assets:
        logger.info(f"Processing asset: {asset_id}")
        export_info = self.export_asset(asset_id, start_time, end_time)
        self.manifest['assets'].append(export_info)

    # Save manifest
    manifest_path = self.output_dir / 'manifest.json'
    with open(manifest_path, 'w') as f:
        json.dump(self.manifest, f, indent=2, default=str)

    logger.info(f"Export complete. Manifest saved to {manifest_path}")

def create_training_dataset(
```

```
        self,
        assets: list = None,
        days: int = 30,
        output_file: str = 'training_dataset.parquet'
    ):
        """Create unified training dataset"""
        if assets is None:
            assets = self.influx.list_assets()

        all_features = []

        for asset_id in assets:
            logger.info(f"Loading features for {asset_id}")
            df = self.influx.download_vibration_features(
                asset_id, f'-{days}d', 'now()'
            )
            if not df.empty:
                df['asset_id'] = asset_id
                all_features.append(df)

        if all_features:
            combined = pd.concat(all_features, ignore_index=True)
            output_path = self.output_dir / output_file
            combined.to_parquet(output_path, index=False)
            logger.info(f"Created training dataset: {output_path} ({len(combined)} rows)")
            return combined

        return pd.DataFrame()


def main():
    parser = argparse.ArgumentParser(description='Unified data export')
    parser.add_argument('--asset', help='Single asset to export')
    parser.add_argument('--all-assets', action='store_true', help='Export all assets')
    parser.add_argument('--start', help='Start time (ISO format or relative)')
    parser.add_argument('--end', default='now()', help='End time')
    parser.add_argument('--days', type=int, default=30, help='Days of history')
    parser.add_argument('--output', required=True, help='Output directory')
    parser.add_argument('--training-dataset', action='store_true',
                        help='Create unified training dataset')

    args = parser.parse_args()

    exporter = UnifiedExporter(args.output)

    if args.training_dataset:
        if args.asset:
            exporter.create_training_dataset(
                assets=[args.asset],
                days=args.days
            )
        else:
            exporter.create_training_dataset(days=args.days)
    elif args.all_assets:
```

```
        exporter.export_all_assets(days=args.days)
    elif args.asset:
        start = args.start or f'-{args.days}d'
        exporter.export_asset(args.asset, start, args.end)
    else:
        parser.print_help()


if __name__ == '__main__':
    main()
```

---

7. DATA IMPORT TOOLS

7.1 Import to InfluxDB

```
#!/usr/bin/env python3
# scripts/import_influxdb.py
"""
Import data to InfluxDB

Usage:
    python import_influxdb.py --file data/sensor_data.csv --bucket sensor_data --measurement sensor_rea
"""

import os
import argparse
from pathlib import Path
import pandas as pd
from influxdb_client import InfluxDBClient, Point, WriteOptions
from influxdb_client.client.write_api import SYNCHRONOUS
from loguru import logger
from datetime import datetime

class InfluxDBImporter:
    """Import data to InfluxDB"""

    def __init__(self):
        self.url = os.environ.get('INFLUXDB_URL')
        self.token = os.environ.get('INFLUXDB_TOKEN')
        self.org = os.environ.get('INFLUXDB_ORG')

        self.client = InfluxDBClient(
            url=self.url,
            token=self.token,
            org=self.org
        )

    def import_csv(
        self,
        file_path: str,
        bucket: str,
        measurement: str,
```

```
    time_column: str = 'timestamp',
    tag_columns: list = None,
    field_columns: list = None,
    batch_size: int = 5000
):
    """
    Import CSV file to InfluxDB.

    Args:
        file_path: Path to CSV file
        bucket: Target bucket
        measurement: Measurement name
        time_column: Column to use as timestamp
        tag_columns: Columns to use as tags
        field_columns: Columns to use as fields (None = all except time/tags)
        batch_size: Batch size for writing
    """
    logger.info(f"Reading {file_path}")
    df = pd.read_csv(file_path, parse_dates=[time_column])

    if tag_columns is None:
        tag_columns = []

    if field_columns is None:
        field_columns = [c for c in df.columns
                         if c not in [time_column] + tag_columns]

    write_api = self.client.write_api(write_options=WriteOptions(
        batch_size=batch_size,
        flush_interval=10_000
    ))

    logger.info(f"Importing {len(df)} rows to {bucket}/{measurement}")

    points = []
    for _, row in df.iterrows():
        point = Point(measurement)

        # Set timestamp
        point = point.time(row[time_column])

        # Set tags
        for tag in tag_columns:
            if pd.notna(row[tag]):
                point = point.tag(tag, str(row[tag]))

        # Set fields
        for field in field_columns:
            if pd.notna(row[field]):
                point = point.field(field, float(row[field]))

        points.append(point)

        if len(points) >= batch_size:
```

```
                write_api.write(bucket=bucket, record=points)
                points = []

        # Write remaining points
        if points:
            write_api.write(bucket=bucket, record=points)

        write_api.close()
        logger.info(f"Import complete")

    def import_parquet(
        self,
        file_path: str,
        bucket: str,
        measurement: str,
        time_column: str = 'timestamp',
        tag_columns: list = None,
        batch_size: int = 5000
    ):
        """Import Parquet file to InfluxDB"""
        df = pd.read_parquet(file_path)

        # Convert to temporary CSV and import
        temp_csv = '/tmp/import_temp.csv'
        df.to_csv(temp_csv, index=False)

        self.import_csv(
            temp_csv, bucket, measurement,
            time_column, tag_columns, batch_size=batch_size
        )

        os.remove(temp_csv)


def main():
    parser = argparse.ArgumentParser(description='Import data to InfluxDB')
    parser.add_argument('--file', required=True, help='Input file (CSV or Parquet)')
    parser.add_argument('--bucket', required=True, help='Target bucket')
    parser.add_argument('--measurement', required=True, help='Measurement name')
    parser.add_argument('--time-column', default='timestamp', help='Time column')
    parser.add_argument('--tag-columns', nargs='+', help='Tag columns')
    parser.add_argument('--batch-size', type=int, default=5000)

    args = parser.parse_args()

    importer = InfluxDBImporter()

    file_path = Path(args.file)
    if file_path.suffix == '.csv':
        importer.import_csv(
            args.file, args.bucket, args.measurement,
            args.time_column, args.tag_columns,
            batch_size=args.batch_size
        )
```

```
    elif file_path.suffix == '.parquet':
        importer.import_parquet(
            args.file, args.bucket, args.measurement,
            args.time_column, args.tag_columns,
            batch_size=args.batch_size
        )
    else:
        logger.error(f"Unsupported file format: {file_path.suffix}")


if __name__ == '__main__':
    main()
```

---

8. DATA TRANSFORMATION

8.1 Data Transformation Pipeline

```
#!/usr/bin/env python3
# scripts/transform_data.py
"""
Data Transformation Pipeline

Usage:
    python transform_data.py --input raw_data/ --output processed_data/ --config transform_config.yaml
"""

import argparse
from pathlib import Path
import pandas as pd
import numpy as np
from scipy import signal, stats
from sklearn.preprocessing import StandardScaler, RobustScaler
from sklearn.impute import KNNImputer
import yaml
from loguru import logger


class DataTransformer:
    """Data transformation pipeline"""

    def __init__(self, config: dict):
        self.config = config
        self.scaler = None
        self.imputer = None

    def remove_outliers(self, df: pd.DataFrame, columns: list = None) -> pd.DataFrame:
        """Remove outliers using IQR method"""
        if columns is None:
            columns = df.select_dtypes(include=[np.number]).columns

        df_clean = df.copy()
```

```
        for col in columns:
            Q1 = df_clean[col].quantile(0.25)
            Q3 = df_clean[col].quantile(0.75)
            IQR = Q3 - Q1

            lower = Q1 - 1.5 * IQR
            upper = Q3 + 1.5 * IQR

            df_clean[col] = df_clean[col].clip(lower, upper)

        return df_clean

    def impute_missing(self, df: pd.DataFrame, method: str = 'knn') -> pd.DataFrame:
        """Impute missing values"""
        numeric_cols = df.select_dtypes(include=[np.number]).columns

        if method == 'knn':
            if self.imputer is None:
                self.imputer = KNNImputer(n_neighbors=5)
            df[numeric_cols] = self.imputer.fit_transform(df[numeric_cols])
        elif method == 'mean':
            df[numeric_cols] = df[numeric_cols].fillna(df[numeric_cols].mean())
        elif method == 'median':
            df[numeric_cols] = df[numeric_cols].fillna(df[numeric_cols].median())
        elif method == 'forward':
            df[numeric_cols] = df[numeric_cols].fillna(method='ffill')

        return df

    def normalize(self, df: pd.DataFrame, method: str = 'standard') -> pd.DataFrame:
        """Normalize numeric columns"""
        numeric_cols = df.select_dtypes(include=[np.number]).columns

        if method == 'standard':
            if self.scaler is None:
                self.scaler = StandardScaler()
            df[numeric_cols] = self.scaler.fit_transform(df[numeric_cols])
        elif method == 'robust':
            if self.scaler is None:
                self.scaler = RobustScaler()
            df[numeric_cols] = self.scaler.fit_transform(df[numeric_cols])
        elif method == 'minmax':
            from sklearn.preprocessing import MinMaxScaler
            if self.scaler is None:
                self.scaler = MinMaxScaler()
            df[numeric_cols] = self.scaler.fit_transform(df[numeric_cols])

        return df

    def resample_timeseries(
        self,
        df: pd.DataFrame,
        time_column: str,
        freq: str,
```

```
    agg_func: str = 'mean'
) -> pd.DataFrame:
    """Resample time series data"""
    df = df.copy()
    df[time_column] = pd.to_datetime(df[time_column])
    df = df.set_index(time_column)

    agg_funcs = {
        'mean': 'mean',
        'sum': 'sum',
        'min': 'min',
        'max': 'max',
        'first': 'first',
        'last': 'last'
    }

    df_resampled = df.resample(freq).agg(agg_funcs.get(agg_func, 'mean'))
    return df_resampled.reset_index()

def apply_filter(
    self,
    df: pd.DataFrame,
    column: str,
    filter_type: str,
    params: dict
) -> pd.DataFrame:
    """Apply signal filter"""
    df = df.copy()

    if filter_type == 'butterworth':
        fs = params.get('sampling_rate', 1000)
        fc = params.get('cutoff', 100)
        order = params.get('order', 4)
        btype = params.get('type', 'low')

        b, a = signal.butter(order, fc / (fs / 2), btype=btype)
        df[column] = signal.filtfilt(b, a, df[column])

    elif filter_type == 'moving_average':
        window = params.get('window', 10)
        df[column] = df[column].rolling(window=window, center=True).mean()

    elif filter_type == 'exponential':
        alpha = params.get('alpha', 0.3)
        df[column] = df[column].ewm(alpha=alpha).mean()

    return df

def create_lag_features(
    self,
    df: pd.DataFrame,
    columns: list,
    lags: list
) -> pd.DataFrame:
```

```
        """Create lagged features"""
        df = df.copy()

        for col in columns:
            for lag in lags:
                df[f'{col}_lag_{lag}'] = df[col].shift(lag)

        return df

    def create_rolling_features(
        self,
        df: pd.DataFrame,
        columns: list,
        windows: list,
        funcs: list = ['mean', 'std', 'min', 'max']
    ) -> pd.DataFrame:
        """Create rolling window features"""
        df = df.copy()

        for col in columns:
            for window in windows:
                for func in funcs:
                    df[f'{col}_rolling_{window}_{func}'] = \
                        df[col].rolling(window=window).agg(func)

        return df

    def transform(self, df: pd.DataFrame) -> pd.DataFrame:
        """Apply full transformation pipeline based on config"""
        steps = self.config.get('steps', [])

        for step in steps:
            step_type = step.get('type')
            params = step.get('params', {})

            logger.info(f"Applying transformation: {step_type}")

            if step_type == 'remove_outliers':
                df = self.remove_outliers(df, **params)
            elif step_type == 'impute':
                df = self.impute_missing(df, **params)
            elif step_type == 'normalize':
                df = self.normalize(df, **params)
            elif step_type == 'resample':
                df = self.resample_timeseries(df, **params)
            elif step_type == 'filter':
                df = self.apply_filter(df, **params)
            elif step_type == 'lag_features':
                df = self.create_lag_features(df, **params)
            elif step_type == 'rolling_features':
                df = self.create_rolling_features(df, **params)

        return df
```

```
def main():
    parser = argparse.ArgumentParser(description='Data transformation pipeline')
    parser.add_argument('--input', required=True, help='Input file or directory')
    parser.add_argument('--output', required=True, help='Output file or directory')
    parser.add_argument('--config', required=True, help='Config YAML file')

    args = parser.parse_args()

    # Load config
    with open(args.config) as f:
        config = yaml.safe_load(f)

    transformer = DataTransformer(config)

    input_path = Path(args.input)
    output_path = Path(args.output)

    if input_path.is_file():
        # Single file
        if input_path.suffix == '.csv':
            df = pd.read_csv(input_path)
        else:
            df = pd.read_parquet(input_path)

        df_transformed = transformer.transform(df)

        if output_path.suffix == '.csv':
            df_transformed.to_csv(output_path, index=False)
        else:
            df_transformed.to_parquet(output_path, index=False)

        logger.info(f"Saved transformed data to {output_path}")

    else:
        # Directory
        output_path.mkdir(parents=True, exist_ok=True)

        for file_path in input_path.glob('*.parquet'):
            df = pd.read_parquet(file_path)
            df_transformed = transformer.transform(df)

            out_file = output_path / file_path.name
            df_transformed.to_parquet(out_file, index=False)
            logger.info(f"Processed {file_path.name}")


if __name__ == '__main__':
    main()
```

8.2 Transformation Config Example

```
# transform_config.yaml
steps:
```

```yaml
  - type: remove_outliers
    params:
      columns: ['temperature', 'vibration', 'pressure']

  - type: impute
    params:
      method: knn

  - type: filter
    params:
      column: vibration
      filter_type: butterworth
      params:
        sampling_rate: 25600
        cutoff: 10000
        order: 4
        type: low

  - type: resample
    params:
      time_column: timestamp
      freq: '1H'
      agg_func: mean

  - type: rolling_features
    params:
      columns: ['temperature', 'vibration', 'pressure']
      windows: [3, 6, 12, 24]
      funcs: ['mean', 'std', 'min', 'max']

  - type: lag_features
    params:
      columns: ['temperature', 'vibration']
      lags: [1, 2, 3, 6, 12, 24]

  - type: normalize
    params:
      method: robust
```

---

9. BACKUP & RESTORE

9.1 Full Backup Script

```bash
#!/bin/bash
# scripts/full_backup.sh
# Complete backup of all data sources

set -e

BACKUP_DIR="${1:-/backup/$(date +%Y%m%d)}"
mkdir -p "$BACKUP_DIR"
```

```
echo "Starting full backup to $BACKUP_DIR"
echo "=========================================="

# 1. PostgreSQL backup
echo "Backing up PostgreSQL..."
pg_dump -h $POSTGRES_HOST -U $POSTGRES_USER -d $POSTGRES_DB \
    -Fc -f "$BACKUP_DIR/postgres_full.dump"
echo "  PostgreSQL backup: $(du -h $BACKUP_DIR/postgres_full.dump | cut -f1)"

# 2. InfluxDB backup
echo "Backing up InfluxDB..."
influx backup "$BACKUP_DIR/influxdb" \
    --host $INFLUXDB_URL \
    --token $INFLUXDB_TOKEN
echo "  InfluxDB backup: $(du -sh $BACKUP_DIR/influxdb | cut -f1)"

# 3. MinIO/S3 sync
echo "Syncing MinIO buckets..."
mc mirror oilgas/ml-data "$BACKUP_DIR/minio/ml-data"
mc mirror oilgas/documents "$BACKUP_DIR/minio/documents"
echo "  MinIO backup: $(du -sh $BACKUP_DIR/minio | cut -f1)"

# 4. Milvus backup (via API)
echo "Backing up Milvus collections..."
python3 << 'EOF'
from pymilvus import connections, utility, Collection
import json

connections.connect(host="milvus.oilgas.internal", port="19530")

collections = utility.list_collections()
backup_info = {
    'collections': [],
    'timestamp': datetime.utcnow().isoformat()
}

for coll_name in collections:
    coll = Collection(coll_name)
    backup_info['collections'].append({
        'name': coll_name,
        'num_entities': coll.num_entities,
        'schema': str(coll.schema)
    })

with open('$BACKUP_DIR/milvus_info.json', 'w') as f:
    json.dump(backup_info, f, indent=2)
EOF

# 5. Create manifest
echo "Creating backup manifest..."
cat > "$BACKUP_DIR/manifest.json" << EOF
{
  "backup_date": "$(date -Iseconds)",
  "hostname": "$(hostname)",
```

```
  "components": {
    "postgresql": {
      "file": "postgres_full.dump",
      "size": "$(du -b $BACKUP_DIR/postgres_full.dump | cut -f1)"
    },
    "influxdb": {
      "directory": "influxdb",
      "size": "$(du -sb $BACKUP_DIR/influxdb | cut -f1)"
    },
    "minio": {
      "directory": "minio",
      "size": "$(du -sb $BACKUP_DIR/minio | cut -f1)"
    }
  }
}
EOF

# 6. Compress backup
echo "Compressing backup..."
tar czf "$BACKUP_DIR.tar.gz" -C "$(dirname $BACKUP_DIR)" "$(basename $BACKUP_DIR)"

# 7. Upload to remote storage (optional)
# aws s3 cp "$BACKUP_DIR.tar.gz" s3://backup-bucket/

echo "============================================"
echo "Backup complete!"
echo "Total size: $(du -h $BACKUP_DIR.tar.gz | cut -f1)"
echo "Location: $BACKUP_DIR.tar.gz"
```

---

10. COMPLETE WORKFLOWS

10.1 ML Training Data Preparation

```
#!/bin/bash
# workflows/prepare_training_data.sh
# End-to-end workflow for preparing ML training data

set -e

# Configuration
OUTPUT_DIR="./training_data_$(date +%Y%m%d)"
DAYS=90
ASSETS="PUMP-001 PUMP-002 COMP-001"

echo "ML Training Data Preparation Pipeline"
echo "======================================"
echo "Output: $OUTPUT_DIR"
echo "Days: $DAYS"
echo "Assets: $ASSETS"

mkdir -p "$OUTPUT_DIR"
```

```
# Step 1: Export sensor data from InfluxDB
echo ""
echo "Step 1: Exporting sensor data from InfluxDB..."
python scripts/influxdb_download.py \
    --asset "$ASSETS" \
    --start "-${DAYS}d" \
    --end "now()" \
    --output "$OUTPUT_DIR/raw" \
    --type features

# Step 2: Export metadata from PostgreSQL
echo ""
echo "Step 2: Exporting metadata from PostgreSQL..."
python scripts/postgres_download.py \
    --preset assets \
    --output "$OUTPUT_DIR/metadata"

# Step 3: Export labels/alerts from PostgreSQL
echo ""
echo "Step 3: Exporting alert labels..."
python scripts/postgres_download.py \
    --query "SELECT * FROM alerts.alert_instances WHERE triggered_at > NOW() - INTERVAL '${DAYS} days'"
    --output "$OUTPUT_DIR/labels/alerts.parquet" \
    --format parquet

# Step 4: Transform and clean data
echo ""
echo "Step 4: Transforming data..."
python scripts/transform_data.py \
    --input "$OUTPUT_DIR/raw" \
    --output "$OUTPUT_DIR/processed" \
    --config configs/transform_config.yaml

# Step 5: Create unified training dataset
echo ""
echo "Step 5: Creating training dataset..."
python scripts/unified_export.py \
    --training-dataset \
    --days $DAYS \
    --output "$OUTPUT_DIR"

# Step 6: Split into train/val/test
echo ""
echo "Step 6: Splitting dataset..."
python << EOF
import pandas as pd
from sklearn.model_selection import train_test_split
from pathlib import Path

output_dir = Path("$OUTPUT_DIR")
df = pd.read_parquet(output_dir / "training_dataset.parquet")

# Split: 70% train, 15% val, 15% test
train, temp = train_test_split(df, test_size=0.3, random_state=42)
```

```
val, test = train_test_split(temp, test_size=0.5, random_state=42)

train.to_parquet(output_dir / "train.parquet", index=False)
val.to_parquet(output_dir / "val.parquet", index=False)
test.to_parquet(output_dir / "test.parquet", index=False)

print(f"Train: {len(train)} samples")
print(f"Val: {len(val)} samples")
print(f"Test: {len(test)} samples")
EOF

# Step 7: Upload to MinIO
echo ""
echo "Step 7: Uploading to MinIO..."
mc cp --recursive "$OUTPUT_DIR/" oilgas/ml-data/training/$(date +%Y%m%d)/

# Step 8: Create dataset manifest
echo ""
echo "Step 8: Creating manifest..."
cat > "$OUTPUT_DIR/dataset_manifest.yaml" << EOF
dataset:
  name: training_$(date +%Y%m%d)
  created: $(date -Iseconds)

source:
  influxdb:
    time_range: "${DAYS} days"
    assets: [$ASSETS]

splits:
  train: $(wc -l < "$OUTPUT_DIR/train.parquet")
  val: $(wc -l < "$OUTPUT_DIR/val.parquet")
  test: $(wc -l < "$OUTPUT_DIR/test.parquet")

transformations:
  - remove_outliers
  - impute_missing
  - normalize

storage:
  local: $OUTPUT_DIR
  remote: s3://ml-data/training/$(date +%Y%m%d)/
EOF

echo ""
echo "======================================"
echo "Pipeline complete!"
echo "Dataset ready at: $OUTPUT_DIR"
echo "Remote: oilgas/ml-data/training/$(date +%Y%m%d)/"
```

10.2 Quick Reference Commands

```
# ============================================
# QUICK REFERENCE: DATA OPERATIONS
```

```
# ============================================

# --- INFLUXDB ---
# Query last hour of data
influx query 'from(bucket:"sensor_data") |> range(start:-1h) |> limit(n:100)'

# Export to CSV
influx query 'from(bucket:"sensor_data") |> range(start:-7d)' --raw > data.csv

# --- POSTGRESQL ---
# Connect
psql -h $POSTGRES_HOST -U $POSTGRES_USER -d $POSTGRES_DB

# Export table to CSV
\copy (SELECT * FROM asset_mgmt.assets) TO 'assets.csv' WITH CSV HEADER;

# Dump schema
pg_dump -h $POSTGRES_HOST -U $POSTGRES_USER -d $POSTGRES_DB --schema-only > schema.sql

# --- MINIO ---
# List buckets
mc ls oilgas

# Upload file
mc cp myfile.parquet oilgas/ml-data/

# Download file
mc cp oilgas/ml-data/model.pt ./

# Sync directory
mc mirror ./local/ oilgas/remote/

# --- KAFKA ---
# List topics
kafka-topics.sh --bootstrap-server $KAFKA_BOOTSTRAP_SERVERS --list

# Consume messages
kcat -b $KAFKA_BOOTSTRAP_SERVERS -t sensor_readings -C -c 10

# Consumer lag
kafka-consumer-groups.sh --bootstrap-server $KAFKA_BOOTSTRAP_SERVERS --describe --group my-group

# --- PYTHON QUICK COMMANDS ---
# Load InfluxDB data
python -c "from influxdb_download import InfluxDBDownloader; d=InfluxDBDownloader(); print(d.list_assets

# Load PostgreSQL data
python -c "from postgres_download import PostgresDownloader; d=PostgresDownloader(); print(d.list_tables
```

END OF DATA DOWNLOAD & MANAGEMENT TOOLS GUIDE